Multi-Agent System Assignment

Part 1: Design & Build a Multi-Agent Application

Objective

Build a multi-agent system that uses **LangGraph** and the **Tavily API** to solve a real-world information task. Your goal is to demonstrate creative design, technical execution, and effective agent collaboration

Project Scope

- **Multi-Agent Architecture**
  Design a LangGraph with agents that each handle a distinct responsibility (e.g., retrieval, synthesis, reasoning). Ensure modularity and clear role separation for scalable collaboration
- **Use Case**
  Define a unique and meaningful application for your system
  Be creative—feel free to explore ideas inspired by Tavily's Use Cases

Requirements

- Use **LangGraph** to coordinate agent workflows and transitions
- Use the **Tavily API** (`search`, `extract`, `crawl`, `map`) to integrate real-time web data
- You'll receive an **OpenAI API key** (with a $10 cap) for LLM integration. Contact us if you run into limits

---

Part 2: Deploy on AWS with MongoDB and a Simple UI

Objective

Deploy your system in a production-ready environment with a working user interface

Requirements

- **Backend**: Host your Python app on **AWS Elastic Beanstalk**, using environment variables and multi-instance support
- **Database Service**: Use **MongoDB Atlas** to store user queries, agent outputs, and metadata. Integrate via the official MongoDB Python client
- **Frontend**: Build a simple, modern web UI (React, Vue, etc.) where users can submit queries, trigger the pipeline, view results, and export outputs
- **Integration**: Ensure full connectivity between Frontend ↔ Backend ↔ MongoDB Atlas
- **Testing**: Validate correctness, error handling, and data logging

---

Demo Requirement

Record a short demo (3–5 minutes) showing:

- The UI in action
- A complete query-to-result flow
- How the agents collaborate
- (Optional) MongoDB/log insights

Publication & Documentation

- **GitHub Repository**
  Include a `README.md` with project summary, setup, usage, example(s), and folder structure. One or two repos (frontend/backend) are both acceptable.
- **Technical Docs**
  Cover architecture, agent roles, LangGraph flow, database schema, and deployment guide.

Evaluation Criteria

Category	Description
Functionality	Agents coordinate effectively to solve the task

Creativity	Originality in use case and agent design
Code Quality	Clean, modular, and maintainable
UI/UX Design	Clear and usable interface
Deployment	Reliable AWS & MongoDB setup
Documentation & Demo	Clear docs and helpful demo recording

Submission Details

Deadline: Within **2 weeks** of receiving this assignment

Deliverables:

- GitHub repository link(s)
- Demo video link (e.g., Loom, YouTube)
- Live app URL (if applicable)
- Access credentials or notes (if needed)